CNR Handwerker Plus REST-API

Sprint 5 · Slim Framework 4 · PHP 8.1+ · MariaDB

v1.0 **Slim 4** **REST · JSON**

- ✅ 6 Ressourcen · 22 Endpoints
- ✅ API-Key + Session Auth
- ✅ Rollensystem (4 Stufen)
- ✅ PDO Prepared Statements
- ✅ Argon2ID Passwort-Hashing
- ✅ CORS-Header

Übersicht

🔗 Basis-URL

```
http://localhost/api
```

Alle Antworten im JSON-Format:

```
Content-Type: application/json; charset=UTF-8
```

🧱 Technologie-Stack

Framework	**Slim Framework 4**
Sprache	**PHP 8.1+**
Datenbank	**MariaDB 10.4+**
DB-Zugriff	**PDO (Prepared Statements)**
Passwort	**Argon2ID / bcrypt**
Webserver	**Apache + mod_rewrite**

🌳 Antwort-Struktur

✅ Erfolgreiche Antwort:

```
{
  "status":  "success",
  "message": "Beschreibung",  // optional
  "data":    { ... }          // Nutzlast
}
```

❌ Fehler-Antwort:

```
{
  "status":  "error",
  "message": "Fehlerbeschreibung",
  "details": { ... }  // optional
}
```

HTTP-Statuscodes

- **200** OK — Erfolgreich
- **201** Created — Ressource erstellt

- **400** Bad Request — Fehlerhafte Eingabe
- **401** Unauthorized — Nicht angemeldet
- **403** Forbidden — Keine Berechtigung
- **404** Not Found — Nicht gefunden
- **409** Conflict — Konflikt (z.B. Trigger)
- **422** Unprocessable — Validierungsfehler

- **500** Internal Server Error
- **503** Service Unavailable (DB-Fehler)

Pagination

Alle Listen-Endpunkte unterstützen Pagination:

```
GET /api/kunden?page=2&limit=10

// Antwort enthält immer:
{
  "total": 42,   // Gesamtanzahl
  "page":  2,   // Aktuelle Seite
  "limit": 10,  // Ergebnisse pro Seite (max. 100)
  "pages": 5    // Gesamtanzahl Seiten
}
```

Installation

1 **Voraussetzungen prüfen**

Folgende Software muss installiert sein:

- PHP 8.1 oder höher
- XAMPP (Apache + MariaDB) oder anderer Webserver
- Composer (PHP Paketmanager)

2 **Projekt-Dateien platzieren**

Den `slim-api` Ordner nach `C:\xampp\htdocs\` kopieren:

```
C:\xampp\htdocs\
└── slim-api\
    ├── composer.json
    ├── config\
    ├── public\        ← Apache DocumentRoot
    └── src\
```

3 **Abhängigkeiten installieren**

Terminal im `slim-api` Ordner öffnen:

```
cd C:\xampp\htdocs\slim-api
composer install
```

Dies erstellt den `vendor/` Ordner mit Slim Framework und allen Abhängigkeiten.

4 **Datenbank konfigurieren**

SQL-Dump einspielen:

```
mysql -u root -p handwerkerpro_db < handwerkerpro_db.sql
```

Dann in `config/database.php` die Zugangsdaten anpassen:

```
return [
    'host' => 'localhost',
    'name' => 'handwerkerpro_db',
    'user' => 'root',
    'pass' => '',  // Dein Passwort
];
```

5 **Apache konfigurieren**

In `C:\xampp\apache\conf\extra\httpd-vhosts.conf` eintragen:

```
<VirtualHost *:80>
    DocumentRoot "C:/xampp/htdocs/slim-api/public"
    ServerName localhost
    <Directory "C:/xampp/htdocs/slim-api/public">
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

In `C:\xampp\apache\conf\httpd.conf` sicherstellen dass diese Zeile aktiv ist:

```
LoadModule rewrite_module modules/mod_rewrite.so
```

6 **Apache neu starten & testen**

XAMPP Control Panel → Apache → Restart

Im Browser öffnen:

```
http://localhost/api
```

Wenn eine JSON-Antwort mit allen Endpoints erscheint, ist die Installation erfolgreich.

7 **Test-Client öffnen**

```
http://localhost/docs/testclient.html
```

Authentifizierung

Die API unterstützt zwei Methoden. **Methode 1 (API-Key)** wird für alle Anfragen empfohlen.

Methode 1: API-Key Header

```
X-API-Key: admin-key-12345

// Alternativ:
Authorization: Bearer admin-key-12345
```

Methode 2: Session-Login

```
POST /api/auth/login
{
  "email":    "[email protected]",
  "password": "passwort"
}
```

Demo API-Keys

API-Key	Mitarbeiter	Rolle	Berechtigungen
`admin-key-12345`	Max Mustermann	**admin**	Vollzugriff
`meister-key-67890`	Lena Zimmer	**meister**	Lesen, Erstellen, Bearbeiten, Löschen (außer Material löschen)
`geselle-key-11111`	Klaus Bauer	**geselle**	Lesen, Erstellen, Bearbeiten

Rollenhierarchie

```
// Zugriffslevel (höher = mehr Rechte)
admin (5) > meister (4) > geselle (3) > azubi (2) > buero (1)

// Wer darf was:
GET    (alle Rollen)  → Daten lesen
POST   (geselle+)    → Neue Einträge erstellen
PUT    (geselle+)    → Einträge bearbeiten
DELETE (meister+)   → Kunden/Aufträge löschen
DELETE (admin)      → Material löschen
```

Fehlerbehandlung & Validierung

Validierungsregeln

- Pflichtfelder werden geprüft → `422`
- ENUM-Werte gegen Whitelist geprüft → `422`
- Datumsformat `YYYY-MM-DD` → `422`
- Numerische Werte auf Positivität → `422`
- E-Mail via `filter_var()` → `422`
- Ressource nicht gefunden → `404`

- DB-Trigger-Fehler (z.B. offene Aufträge) → `409`

Beispiel 422-Antwort

```
{
  "status":  "error",
  "message": "Pflichtfelder fehlen.",
  "details": {
    "fehlende_felder": [
      "vorname",
      "nachname"
    ]
  }
}
```

Sicherheit

Prepared Statements

Alle DB-Abfragen über PDO Prepared Statements. String-Concatenation für SQL ist vollständig verboten.

Passwort-Hashing

Argon2ID (Standard) oder bcrypt via `password_hash()`. Passwörter werden niemals im Klartext gespeichert.

Input-Sanitizing

Alle Eingaben werden via `strip_tags()` und `trim()` bereinigt (XSS-Schutz).

CORS-Header

CorsMiddleware setzt alle notwendigen Header. In Produktion sollte `Allow-Origin` eingeschränkt werden.

Auth Endpoints

POST `/api/auth/login` **Öffentlich**

Meldet einen Mitarbeiter an und startet eine Session.

Request Body

Feld	Typ	Pflicht	Beschreibung
`email`	string	Pflicht	E-Mail-Adresse des Mitarbeiters
`password`	string	Pflicht	Passwort (Demo: `demo123`)

Beispiel-Aufruf

```
curl -X POST http://localhost/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email":    "[email protected]",
    "password": "demo123"
  }'
```

Antwort `200`

```
{
  "status":  "success",
  "message": "Login erfolgreich.",
  "data": {
    "mitarbeiter_id": 1,
    "vorname": "Max",
    "rolle":   "admin",
    "session_aktiv": true
  }
}
```

Fehler: `401` falsches Passwort · `422` E-Mail/Passwort fehlt

GET /api/auth/me

alle Rollen

Gibt Informationen über den aktuell authentifizierten Benutzer zurück.

```
curl http://localhost/api/auth/me \
  -H "X-API-Key: admin-key-12345"
```

POST /api/auth/logout

alle Rollen

Beendet die aktuelle Session.

```
curl -X POST http://localhost/api/auth/logout \
  -H "X-API-Key: admin-key-12345"
```

Dashboard

GET `/api/dashboard` **alle Rollen**

Vollständige Dashboard-Übersicht mit KPIs, heutigen Terminen, Notfall-Aufträgen und kritischen Lagerbeständen.

Beispiel-Aufruf

```
curl http://localhost/api/dashboard \
  -H "X-API-Key: admin-key-12345"
```

Enthaltene Daten

- `kpis` — Kunden gesamt, offene Aufträge, überfällige Rechnungen, offener Betrag, kritische Lagerbestände
- `auftraege_statistik` — Anzahl pro Status
- `rechnungen_statistik` — Anzahl pro Status + offener Betrag
- `termine_heute` — Alle Termine des heutigen Tages
- `notfall_auftraege` — Offene Notfall/Dringend Aufträge
- `lager_kritisch` — Materialien mit Bestand < 5
- `wartende_kunden` — Kunden mit Status "neu" (Top 5)
- `mitarbeiter_auslastung` — Geplante Stunden diese Woche

Kunden

GET `/api/kunden` **alle Rollen**

Query-Parameter

Parameter	Typ		Beschreibung	Beispiel
`page`	int	Optional	Seitennummer (Standard: 1)	`?page=2`
`limit`	int	Optional	Ergebnisse pro Seite, max. 100 (Standard: 25)	`?limit=10`
`typ`	string	Optional	`privat` oder `firma`	`?typ=firma`
`stammkunde`	bool	Optional	`true` oder `false`	`?stammkunde=true`
`search`	string	Optional	Suche in Name / Firmenname	`?search=Müller`

```
curl "http://localhost/api/kunden?typ=firma&stammkunde=true&limit=5" \
  -H "X-API-Key: admin-key-12345"
```

GET `/api/kunden/{id}` **alle Rollen**

Vollständiger Kundendatensatz inkl. Kontakte, Adressen und Personen/Firmen-Details.

```
curl http://localhost/api/kunden/1 \
  -H "X-API-Key: admin-key-12345"
```

GET /api/kunden/{id}/auftraege

alle Rollen

Alle Aufträge eines bestimmten Kunden.

```
curl http://localhost/api/kunden/1/auftraege \
  -H "X-API-Key: admin-key-12345"
```

POST /api/kunden

geselle+

Privatkunde anlegen

Feld	Typ	
`typ`	string	Pflicht `"privat"`
`vorname`	string	Pflicht
`nachname`	string	Pflicht
`ist_stammkunde`	bool	Optional
`notizen`	string	Optional
`kontakt_typ`	string	Optional
`kontakt_wert`	string	Optional

```
curl -X POST http://localhost/api/kunden \
  -H "X-API-Key: admin-key-12345" \
  -H "Content-Type: application/json" \
  -d '{
    "typ":          "privat",
    "vorname":      "Hans",
    "nachname":     "Maier",
    "kontakt_typ":  "Mobil",
    "kontakt_wert": "0170-1234567"
  }'
```

Firmenkunde anlegen

Feld	Typ	
`typ`	string	Pflicht `"firma"`
`firmenname`	string	Pflicht
`ansprechpartner`	string	Optional
`ust_id`	string	Optional

```
curl -X POST http://localhost/api/kunden \
  -H "X-API-Key: admin-key-12345" \
  -H "Content-Type: application/json" \
  -d '{
    "typ":              "firma",
    "firmenname":      "Muster GmbH",
    "ansprechpartner": "Herr Koch",
    "ust_id":          "DE123456789"
  }'
```

Erlaubte kontakt_typ Werte: `Telefon` · `Email` · `Mobil` · `Whatsapp Business`

PUT `/api/kunden/{id}` — geselle+

Felder aktualisieren. Nur gesendete Felder werden geändert.

```
curl -X PUT http://localhost/api/kunden/1 \
  -H "X-API-Key: admin-key-12345" \
  -H "Content-Type: application/json" \
  -d '{ "ist_stammkunde": true, "notizen": "VIP-Kunde" }'
```

DELETE `/api/kunden/{id}` — meister+

Löscht einen Kunden. DB-Trigger `tr_kunden_loeschschutz` verhindert das Löschen bei offenen Aufträgen → `409` .

```
curl -X DELETE http://localhost/api/kunden/5 \
  -H "X-API-Key: meister-key-67890"
```

Aufträge

GET `/api/auftraege` — alle Rollen

Parameter	Typ		Beschreibung
`status`	string	Optional	Kommagetrennt: `neu,aktiv,geplant,abgeschlossen,abgerechnet,storniert`
`prioritaet`	string	Optional	`niedrig` · `normal` · `dringend` · `notfall`
`kunden_id`	int	Optional	Nur Aufträge eines Kunden
`datum_von`	date	Optional	Von-Datum (YYYY-MM-DD)
`datum_bis`	date	Optional	Bis-Datum (YYYY-MM-DD)
`search`	string	Optional	Suche im Titel
`sort`	string	Optional	`erstellt_am` · `prioritaet` · `status` · `auftrag_nr`
`dir`	string	Optional	`ASC` oder `DESC` (Standard: DESC)

```
curl "http://localhost/api/auftraege?status=neu,aktiv&prioritaet=notfall" \
  -H "X-API-Key: admin-key-12345"
```

GET `/api/auftraege/{id}` — alle Rollen

Vollständiger Auftrag inkl. Positionen, Materialien, Termine und berechneter Netto/Brutto-Summe.

```
curl http://localhost/api/auftraege/1 \
  -H "X-API-Key: admin-key-12345"
```

POST `/api/auftraege` **geselle+**

Legt neuen Auftrag an via Stored Procedure `sp_neuer_auftrag`. Auftragsnummer wird automatisch via Trigger generiert.

Feld	Typ		Beschreibung
`kunden_id`	int	Pflicht	ID des Kunden
`titel`	string	Pflicht	Min. 3, max. 150 Zeichen
`prioritaet`	string	Optional	Standard: `normal`
`beschreibung`	string	Optional	Detailbeschreibung

```
curl -X POST http://localhost/api/auftraege \
  -H "X-API-Key: geselle-key-11111" \
  -H "Content-Type: application/json" \
  -d '{
    "kunden_id":   1,
    "titel":       "Heizungsreparatur Keller",
    "prioritaet":  "dringend",
    "beschreibung":"Heizung ausgefallen seit gestern"
  }'
```

PUT `/api/auftraege/{id}/status` **geselle+**

Status ändern. Spezialverhalten:

- `abgeschlossen` → ruft `sp_auftrag_abschliessen` auf → Rechnung wird automatisch erstellt
- `storniert` → ruft `sp_auftrag_stornieren` auf → Materialien werden ins Lager zurückgebucht

```
curl -X PUT http://localhost/api/auftraege/3/status \
  -H "X-API-Key: admin-key-12345" \
  -H "Content-Type: application/json" \
  -d '{ "status": "abgeschlossen" }'
```

DELETE `/api/auftraege/{id}` **meister+**

Löscht Auftrag. Bereits abgerechnete Aufträge können nicht gelöscht werden → `409` .

```
curl -X DELETE http://localhost/api/auftraege/15 \
  -H "X-API-Key: meister-key-67890"
```

🧾 Rechnungen

GET `/api/rechnungen` — alle Rollen

Parameter	Typ		Beschreibung
`status`	string	Optional	`entwurf` · `gesendet` · `bezahlt` · `überfällig`
`kunden_id`	int	Optional	Nur Rechnungen eines Kunden
`ueberfaellig`	bool	Optional	`true` → nur überfällige Rechnungen
`datum_von`	date	Optional	Von-Datum (YYYY-MM-DD)
`datum_bis`	date	Optional	Bis-Datum (YYYY-MM-DD)

```
curl "http://localhost/api/rechnungen?ueberfaellig=true" \
  -H "X-API-Key: admin-key-12345"
```

GET `/api/rechnungen/{id}` — alle Rollen

Einzelne Rechnung mit allen Positionen, Brutto/Netto-Betrag und Verzugstagen.

```
curl http://localhost/api/rechnungen/2 \
  -H "X-API-Key: admin-key-12345"
```

POST `/api/rechnungen/{id}/bezahlen` — alle Rollen

Markiert eine Rechnung als bezahlt via `sp_rechnung_bezahlen`. Bereits bezahlte Rechnung → `409` .

```
curl -X POST http://localhost/api/rechnungen/2/bezahlen \
  -H "X-API-Key: admin-key-12345"
```

PUT `/api/rechnungen/{id}` — geselle+

Status oder Fälligkeitsdatum aktualisieren.

```
curl -X PUT http://localhost/api/rechnungen/2 \
  -H "X-API-Key: admin-key-12345" \
  -H "Content-Type: application/json" \
```

```
  -d '{ "status": "gesendet", "faellig_am": "2026-04-01" }'
```

Material

GET `/api/material` — alle Rollen

Parameter	Typ		Beschreibung
`search`	string	Optional	Suche im Materialnamen
`niedrig_bestand`	—	Optional	Nur Materialien mit Bestand < 5
`bestellliste`	—	Optional	Materialien deren Bestand für Aufträge nicht reicht
`nachbestellen`	—	Optional	View: kritische Bestände (< 5)
`meistverwendet`	—	Optional	Meist verwendete Materialien

```
# Alle Materialien
curl http://localhost/api/material \
  -H "X-API-Key: admin-key-12345"

# Nur kritische Bestände
curl "http://localhost/api/material?niedrig_bestand=1" \
  -H "X-API-Key: admin-key-12345"

# Bestellliste (zu wenig für Aufträge)
curl "http://localhost/api/material?bestellliste=1" \
  -H "X-API-Key: admin-key-12345"
```

POST `/api/material` — meister+

Feld	Typ		Beschreibung
`name`	string	Pflicht	Materialbezeichnung
`preis_pro_einheit`	float	Pflicht	Preis > 0
`beschreibung`	string	Optional	Kurzbeschreibung
`einheit`	string	Optional	Standard: `Stück`
`lagerbestand`	int	Optional	Standard: 0, darf nicht negativ sein

```
curl -X POST http://localhost/api/material \
  -H "X-API-Key: meister-key-67890" \
  -H "Content-Type: application/json" \
  -d '{
    "name":              "Kupferrohr 22mm",
    "beschreibung":     "Heizungsrohr",
    "einheit":          "Meter",
    "lagerbestand":     50,
    "preis_pro_einheit": 15.90
  }'
```

POST /api/material/{id}/nachbestellen

geselle+

Erhöht den Lagerbestand via `sp_material_nachbestellen`.

```
curl -X POST http://localhost/api/material/4/nachbestellen \
  -H "X-API-Key: geselle-key-11111" \
  -H "Content-Type: application/json" \
  -d '{ "menge": 20 }'
```

DELETE /api/material/{id}

admin

Löscht Material. Schlägt fehl wenn Material noch in Aufträgen verwendet wird → `409` .

```
curl -X DELETE http://localhost/api/material/6 \
  -H "X-API-Key: admin-key-12345"
```